

Distributed File Storage System

Semester Project Documentation

Project title

Design and Implementation of a Fault-Tolerant Distributed File Storage System Using Chunking, Deduplication, Replication, and Integrity Verification

Team document

This document describes the project idea, architecture, requirements, database design, APIs, implementation plan, testing strategy, storage optimization, and presentation structure.

1. Project overview

A distributed file storage system stores one file across multiple computers or storage nodes instead of keeping the complete file on one machine.

When a user uploads a file, the system:

1. Accepts the file from the client.
2. Divides it into smaller chunks.
3. Calculates a hash for every chunk.
4. Checks whether an identical chunk already exists.
5. Stores new chunks on different storage nodes.
6. Stores file metadata in a metadata database.
7. Creates backup copies or parity chunks for fault tolerance.

When the user downloads the file, the system:

1. Finds the file metadata.
2. Locates all required chunks.
3. Downloads the chunks from storage nodes.
4. Verifies every chunk.
5. Arranges chunks according to their sequence number.
6. Reconstructs the original file.

The project is inspired by the block-based design of distributed systems such as HDFS. In HDFS, a file is divided into blocks, blocks are stored on DataNodes, and a NameNode manages metadata and replication. [HDFS documentation](#)

2. Problem statement

Traditional file storage keeps a complete file on one computer. This creates several problems:

- A disk failure can make the file unavailable.
- A single computer may not have enough storage.
- Large files are difficult to distribute and process.
- The system may become unavailable when the storage computer is offline.
- Repeated files waste storage space.
- Corrupted data may not be detected automatically.

The proposed system solves these problems by distributing chunks among multiple nodes, maintaining metadata, checking data integrity, and recovering data after a node failure.

3. Objectives

The objectives of the project are:

- To build a client-server distributed file storage prototype.
- To divide files into fixed-size chunks.
- To distribute chunks among several storage nodes.
- To reconstruct the exact original file during download.
- To use SHA-256 hashes for integrity verification.
- To reduce storage waste through chunk-level deduplication.
- To tolerate a storage-node failure using replication.
- To monitor node availability using heartbeat requests.
- To measure storage overhead, upload time, download time, and recovery time.
- To provide a clear demonstration of distributed-system concepts.

4. Scope of the project

Included in the first version

- Three storage nodes running as independent Python processes.
- One metadata server.
- One client application.
- File upload and download.
- Fixed-size chunking, for example 1 MB per chunk.
- SHA-256 hashing.
- Chunk deduplication.

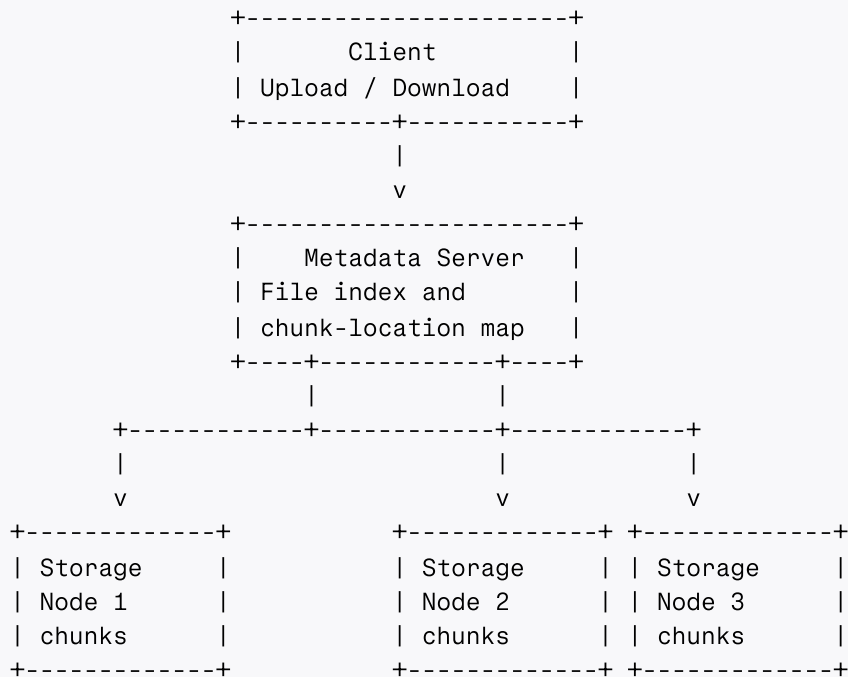
- Replication factor two.
- Node health checking.
- Corruption detection.
- Basic failure recovery.
- Storage and performance reports.

Optional advanced features

- Content-defined chunking.
- Reed-Solomon erasure coding.
- User login and file permissions.
- File encryption.
- Web interface.
- Parallel chunk upload and download.
- Load-based node selection.

Do not start with every advanced feature. First complete upload, download, metadata, hashing, and node distribution. Then add deduplication and replication.

5. System architecture



Components

Component	Responsibility
Client	Selects files, sends upload requests, requests downloads
Metadata server	Stores file metadata, chunk sequence, hashes, and locations
Storage node	Stores, retrieves, verifies, and deletes chunks
Scheduler	Selects nodes using round-robin or load-aware placement
Deduplication module	Reuses an existing chunk instead of storing it again
Replication module	Maintains a backup copy on another node
Recovery module	Recreates missing replicas after failure
Monitoring module	Checks whether nodes are alive

6. Complete upload workflow

Assume the user uploads `research.pdf`.

Step 1: Create a file identifier

Calculate:

```
file_id = SHA256(original_file)
```

The file ID identifies the complete file.

Step 2: Split the file

Use a fixed chunk size during the initial implementation:

```
chunk_size = 1,048,576 bytes = 1 MB
```

A 5.2 MB file becomes six chunks. The last chunk may be smaller.

```
chunk_0000  
chunk_0001  
chunk_0002  
chunk_0003  
chunk_0004  
chunk_0005
```

Step 3: Hash every chunk

For each chunk:

```
chunk_hash = SHA256(chunk_data)
```

The hash is used as the chunk identifier. A chunk can be stored using a path such as:

```
storage/node1/ab/ab12cd89...
```

Step 4: Ask the metadata server whether the chunk exists

If the hash already exists, the system reuses it. If it does not exist, the client uploads the chunk to a selected node.

Step 5: Select storage nodes

For the first prototype, use round-robin placement:

```
chunk 0 -> Node 1  
chunk 1 -> Node 2  
chunk 2 -> Node 3  
chunk 3 -> Node 1
```

For a stronger version, choose the least-used healthy node.

Step 6: Store a replica

With replication factor two:

```
chunk 0 -> Node 1 primary, Node 2 replica  
chunk 1 -> Node 2 primary, Node 3 replica
```

Never place both replicas on the same node.

Step 7: Save metadata

After successful storage, save the file and chunk information in the metadata database.

Step 8: Return an upload result

```
{  
  "success": true,  
  "file_id": "a91f...",  
  "file_name": "research.pdf",  
  "number_of_chunks": 6,  
  "deduplicated_chunks": 2,
```

```
"stored_chunks": 4
}
```

7. Complete download workflow

1. The client sends a file ID to the metadata server.
2. The metadata server returns the file record and ordered chunk list.
3. The client requests each chunk from its primary node.
4. If the primary node is unavailable, the client requests the replica.
5. The client calculates the received chunk hash.
6. If the hash is incorrect, the chunk is marked corrupted and the replica is tried.
7. Chunks are sorted by `sequence_number`.
8. Chunks are written sequentially into the output file.
9. The complete file hash is calculated.
10. The result is compared with the original file hash.

The download is successful only when:

```
SHA256(reconstructed_file) == original_file_hash
```

This proves that the downloaded file is identical to the uploaded file.

8. Storage optimization

8.1 Chunk-level deduplication

Deduplication avoids storing the same chunk multiple times. The metadata server keeps a `reference_count` for every chunk.

Example:

```
File A: chunk A, chunk B, chunk C, chunk D
File B: chunk A, chunk B, chunk X, chunk D
```

Only chunk X needs new physical storage. The other chunks are referenced by both files.

When a file is deleted, decrease the reference count of each chunk. Delete a physical chunk only when its count reaches zero.

8.2 Replication overhead

If a file is stored with replication factor two:

```
physical storage  $\approx 2 \times$  logical file size
```

This is simple and reliable, but it uses more space.

8.3 Erasure coding option

An erasure-coded design uses K data chunks and M coding or parity chunks. The original data can be recovered from any K chunks out of the total $K + M$ chunks. Ceph documentation describes K as the number of data chunks and M as the number of coding chunks; M also represents the number of missing chunks that can be tolerated without data loss. [Ceph documentation](#)

For a (4,2) design:

```
4 data chunks + 2 parity chunks = 6 chunks
```

Storage overhead:

```
(4 + 2) / 4 = 1.5 times the original data
```

This saves space compared with two or three complete replicas, but implementation is more difficult. Use it as an optional advanced module after the replicated system works.

8.4 Content-defined chunking option

Fixed-size chunking is easy to implement, but inserting data near the beginning of a file can change the boundaries of many later chunks. Content-defined chunking selects boundaries according to the content and is widely studied for deduplication systems. The FastCDC research paper discusses CDC and the boundary-shift problem in data deduplication. [FastCDC paper](#)

For the semester MVP, use fixed-size chunks and mention content-defined chunking as future work.

9. Database design

Use SQLite for the prototype.

files **table**

```
CREATE TABLE files (  
    file_id TEXT PRIMARY KEY,  
    file_name TEXT NOT NULL,  
    file_size INTEGER NOT NULL,  
    file_hash TEXT NOT NULL,  
    chunk_size INTEGER NOT NULL,  
    created_at TEXT NOT NULL  
);
```

chunks **table**

```
CREATE TABLE chunks (  
    chunk_hash TEXT PRIMARY KEY,  
    chunk_size INTEGER NOT NULL,  
    reference_count INTEGER DEFAULT 0  
);
```

file_chunks **table**

```
CREATE TABLE file_chunks (  
    file_id TEXT,  
    sequence_number INTEGER,  
    chunk_hash TEXT,  
    PRIMARY KEY (file_id, sequence_number)  
);
```

nodes **table**

```
CREATE TABLE nodes (  
    node_id TEXT PRIMARY KEY,  
    host TEXT NOT NULL,  
    port INTEGER NOT NULL,  
    status TEXT NOT NULL,  
    used_bytes INTEGER DEFAULT 0,  
    last_heartbeat TEXT  
);
```

chunk_locations **table**

```
CREATE TABLE chunk_locations (  
    chunk_hash TEXT,  
    node_id TEXT,  
    copy_type TEXT,  
    status TEXT,
```

```
PRIMARY KEY (chunk_hash, node_id)
);
```

10. Suggested project directory

```
distributed_storage/
├── client/
│   ├── client.py
│   └── gui.py
├── metadata_server/
│   ├── server.py
│   ├── database.py
│   └── scheduler.py
├── storage_node/
│   ├── node_server.py
│   └── node_storage.py
├── common/
│   ├── hashing.py
│   ├── chunking.py
│   └── config.py
├── tests/
│   ├── test_chunking.py
│   ├── test_upload.py
│   └── test_recovery.py
├── storage/
│   ├── node1/
│   ├── node2/
│   └── node3/
├── requirements.txt
└── README.md
```

11. Storage-node API

Implement these endpoints:

Method	Endpoint	Purpose
POST	/store	Store a chunk
GET	/retrieve/{chunk_hash}	Return a chunk
GET	/exists/{chunk_hash}	Check whether a chunk exists
DELETE	/delete/{chunk_hash}	Delete an unused chunk
GET	/health	Return node health
GET	/stats	Return storage statistics

A storage node must verify the hash before accepting data:

```
SHA256(received_data) == supplied_chunk_hash
```

If the values differ, return an error and do not save the chunk.

12. Metadata-server API

Method	Endpoint	Purpose
POST	/files/upload	Register an upload and return placement instructions
POST	/files/complete	Confirm that all chunks were stored
GET	/files	List the user's files
GET	/files/{file_id}	Return metadata and chunk locations
DELETE	/files/{file_id}	Delete a file and update references
GET	/nodes	Display node status
POST	/rebalance	Move chunks to balance storage

13. Node failure and recovery

Failure detection

The metadata server sends a heartbeat request to every node periodically:

```
GET /health
```

If a node fails to respond after a configured number of attempts, mark it as FAILED.

Recovery procedure

1. Find all chunks whose location includes the failed node.
2. Check whether another copy exists.
3. Select a healthy node with sufficient free space.
4. Copy the surviving chunk to the new node.
5. Verify the copied chunk hash.
6. Insert the new location in chunk_locations.
7. Mark the system as healthy again.

Example

```
Before failure:  
chunk A -> Node 1 and Node 2  
  
Node 1 fails:  
chunk A -> Node 2 only  
  
Recovery:  
chunk A -> Node 2 and Node 3
```

The user can still download the file during recovery if at least one valid copy remains.

14. Security and correctness

Implement these protections:

- Use SHA-256 to detect corruption.
- Validate file names and do not allow path traversal such as ../../file.
- Give each user an ownership record.
- Permit downloads only for authorized users.
- Do not trust a client-provided hash without verifying the actual bytes.
- Write uploaded chunks to a temporary file before renaming them to their final hash path.
- Use database transactions when updating reference counts.
- Do not delete a chunk while another upload is using it.

Encryption can be added later. For the basic project, document it as future work unless the team has enough time to implement key management safely.

15. Testing plan

Functional tests

- Upload a small text file.
- Upload a file larger than one chunk.
- Download the file and compare it byte by byte with the original.
- Upload the same file twice and verify deduplication.
- Upload two files with common content and verify chunk reuse.
- Delete a file and verify reference counts.

Failure tests

- Stop one storage node and download a replicated file.
- Stop one node during upload.
- Corrupt a stored chunk manually.
- Remove a primary chunk and verify replica use.
- Recover a failed node and verify rebalancing.

Performance tests

Test files of 1 MB, 10 MB, 50 MB, and 100 MB. Record:

- Upload time.
- Download time.
- Number of chunks.
- Number of deduplicated chunks.
- Logical file size.
- Physical storage size.
- Recovery time.

Correctness assertion

```
assert sha256(original_file) == sha256(downloaded_file)
```

16. Suggested experiment table

Experiment	Logical size	Replication	Physical size	Upload time	Download time	Recovery time
One file, no duplicate	100 MB	1	Record result	Record result	Record result	Not applicable
One file, two replicas	100 MB	2	Record result	Record result	Record result	Record result
Same file uploaded twice	200 MB logical	2	Record result	Record result	Record result	Record result
Erasure-coded version	100 MB	(4, 2)	Record result	Record result	Record result	Record result

Do not invent values. Run the experiments and fill in the actual measurements.

17. Team division of work

Team member	Responsibility
Member 1	Client interface and upload/download workflow
Member 2	Metadata server and SQLite database
Member 3	Storage-node server and local chunk storage
Member 4	Deduplication, replication, and recovery
Member 5	Testing, experiment results, report, and presentation

If the team has fewer members, combine the client and testing responsibilities.

Use Git for collaboration. Create separate branches such as:

```
feature/client  
feature/metadata  
feature/storage-node  
feature/recovery
```

Merge only tested code into the main branch.

18. Development schedule

Week 1: Analysis and design

- Finalize requirements.
- Draw architecture diagrams.
- Design database tables.
- Decide chunk size and replication factor.

Week 2: Chunking and hashing

- Implement file splitting.
- Implement file reconstruction.
- Implement SHA-256 verification.

Week 3: Storage nodes

- Build one storage-node API.
- Run three node processes.
- Add health and statistics endpoints.

Week 4: Metadata server

- Implement file and chunk tables.
- Implement node selection.
- Implement upload metadata registration.

Week 5: Complete upload and download

- Connect client, metadata server, and storage nodes.
- Test multiple chunks.
- Verify reconstructed files.

Week 6: Deduplication and replication

- Add chunk existence checks.
- Add reference counting.
- Store replicas on different nodes.

Week 7: Failure recovery

- Add heartbeat monitoring.
- Simulate node failure.
- Recover chunks to healthy nodes.

Week 8: Testing and documentation

- Perform experiments.
- Prepare graphs and screenshots.
- Complete report and presentation.

19. Limitations

The semester prototype will have limitations:

- SQLite metadata is not suitable for a very large production cluster.
- Running all nodes on one laptop does not provide physical hardware failure isolation.
- Fixed-size chunks are less efficient than content-defined chunks for modified files.
- Replication consumes more storage than erasure coding.
- A single metadata server can become a single point of failure.
- The prototype may not support extremely large files or many simultaneous users.

These limitations are acceptable if they are clearly documented. Future versions could use a replicated metadata service, distributed databases, real machines, encryption, and erasure

coding.

20. Expected outcomes

At the end of the project, the system should demonstrate that:

- A file can be split and distributed among multiple nodes.
- The original file can be reconstructed correctly.
- Duplicate chunks are not stored repeatedly.
- A failed node does not immediately make a replicated file unavailable.
- Corrupted chunks can be detected by hash comparison.
- The system can report storage overhead and recovery time.

21. Viva questions and answers

Why divide a file into chunks?

Chunks make it possible to distribute storage, upload parts independently, recover individual parts, and process large files without loading the complete file into memory.

Why is a metadata server required?

The system needs a mapping between a logical file and its physical chunks. The metadata server stores chunk order, hashes, node locations, file size, and ownership.

What happens if a node fails?

The client uses another replica. The recovery service later creates a new replica on a healthy node.

How do you know that a chunk is not corrupted?

The system calculates SHA-256 on the received bytes and compares it with the stored chunk hash.

How does deduplication save space?

Identical chunks are stored once and referenced by multiple files. The system maintains a reference count and deletes the chunk only when no file references it.

Why not store three copies?

Three replicas provide strong availability but consume approximately three times the logical storage. Two replicas are adequate for the prototype, while erasure coding can reduce overhead further.

What is the advantage of erasure coding?

It provides fault tolerance using parity chunks while requiring less physical storage than complete replicas. In a $(4,2)$ design, six total chunks are stored and any four valid chunks can reconstruct the data.

Is this a real distributed system if all nodes run on one laptop?

It is a distributed-system simulation because the components run as independent network services and communicate through APIs. A stronger demonstration can run the nodes on separate computers or virtual machines.

22. Final recommended implementation

The recommended final version is:

```
Python + FastAPI
1 metadata server
3 storage nodes
SQLite metadata database
1 MB fixed-size chunks
SHA-256 chunk and file hashes
Content-addressed chunk storage
Chunk-level deduplication
Replication factor 2
Heartbeat monitoring
Node failure simulation
Automatic replica recovery
Tkinter or web client
```

Implement erasure coding only if the replicated version is stable. The project will already be strong because it demonstrates partitioning, naming, placement, replication, fault detection, recovery, integrity, and storage optimization.

23. One-paragraph project summary

This project presents a fault-tolerant distributed file storage system in which uploaded files are divided into fixed-size chunks and distributed among multiple storage nodes. A metadata server records file information, chunk order, hashes, and node locations. SHA-256 hashing is used for integrity verification and chunk-level deduplication prevents identical chunks from occupying storage more than once. Each chunk is replicated on a second node so that the file remains downloadable when one node fails. The system detects failed nodes through heartbeat

messages and automatically creates replacement replicas. The prototype will be evaluated using storage overhead, deduplication ratio, upload time, download time, integrity detection, and failure-recovery time.